

Class 6: R functions

Alejandro Ostos (PID: A17978684)

All functions in R have at least 3 things:

- A **name**, we pick this and use it to call the function.
- Input **arguments**, there can be multiple comma separated inputs to the function.
- The **body**, lines of R code that do the work of the function.

Our first wee function:

```
add <- function(x, y = 1, z = 1){  
  (x + y)/z  
}
```

Let's test our function:

```
add(10, 1, 34)
```

```
[1] 0.3235294
```

A second function

Let's try something more interesting. Make a sequence generation tool.

The `sample()` function could be useful here.

```
sample(1:10, 3)
```

```
[1] 1 6 7
```

change this to work with the nucleotides A, C, G and T. Return 3 of them.

```
n <- c("A", "C", "G", "T")
sample(n, 5, T)
```

```
[1] "C" "A" "T" "T" "G"
```

Turn this snippet into a function that returns a user specified length DNA sequence. Let's call it `generate_dna()`...

```
generate_dna <- function(y, fasta = T) {
  output <- sample(c("A", "C", "G", "T"), y, T)
  toutput <- paste(output, collapse = "")
  cat("Well done you!\n")
  if(fasta){
    return(toutput)
  } else{
    return(output)
  }
}

generate_dna(50)
```

Well done you!

```
[1] "ATGGATAGGGGATCAACGAAATTTTGGTAGTTTTCGTACCGGGGGTGTCC"
```

```
generate_dna(30, F)
```

Well done you!

```
[1] "A" "C" "T" "C" "C" "G" "C" "C" "A" "G" "T" "C" "C" "C" "A" "G" "T" "G" "G"
[20] "T" "A" "G" "C" "A" "A" "A" "A" "T" "C" "A"
```

The next code assigns the aminoacid to the corresponding codon (I did it for fun).

```

codon_table <- list(
  "ATA"="I", "ATC"="I", "ATT"="I", "ATG"="M",
  "ACA"="T", "ACC"="T", "ACG"="T", "ACT"="T",
  "AAC"="N", "AAT"="N", "AAA"="K", "AAG"="K",
  "AGC"="S", "AGT"="S", "AGA"="R", "AGG"="R",
  "CTA"="L", "CTC"="L", "CTG"="L", "CTT"="L",
  "CCA"="P", "CCC"="P", "CCG"="P", "CCT"="P",
  "CAC"="H", "CAT"="H", "CAA"="Q", "CAG"="Q",
  "CGA"="R", "CGC"="R", "CGG"="R", "CGT"="R",
  "GTA"="V", "GTC"="V", "GTG"="V", "GTT"="V",
  "GCA"="A", "GCC"="A", "GCG"="A", "GCT"="A",
  "GAC"="D", "GAT"="D", "GAA"="E", "GAG"="E",
  "GGA"="G", "GGC"="G", "GGG"="G", "GGT"="G",
  "TCA"="S", "TCC"="S", "TCG"="S", "TCT"="S",
  "TTC"="F", "TTT"="F", "TTA"="L", "TTG"="L",
  "TAC"="Y", "TAT"="Y", "TAA"="*", "TAG"="*",
  "TGC"="C", "TGT"="C", "TGA"="*", "TGG"="W"
  #Table of aminoacids linked to a single character
)
translate_seq <- function(dna_seq) {
  #The following line determines where each codon starts and ends.
  codons <- substring(dna_seq, seq(1, nchar(dna_seq)-2, 3), seq(3, nchar(dna_seq), 3))
  aa <- sapply(codons, function(codon) codon_table[[toupper(codon)]])
  paste(aa, collapse = "")
}

translate_seq(generate_dna(36))

```

Well done you!

```
[1] "VSMEVSLEE*FV"
```

The next function gives you each of the names of the aminoacids present

```

translate_aminoacids <- function(aa_sequence) {
  # Dictionary: single character into full name
  aa_names <- list(
    "A"="Alanine", "R"="Arginine", "N"="Asparagine", "D"="Aspartic acid",
    "C"="Cysteine", "Q"="Glutamine", "E"="Glutamic acid", "G"="Glycine",
    "H"="Histidine", "I"="Isoleucine", "L"="Leucine", "K"="Lysine",
    "M"="Methionine", "F"="Phenylalanine", "P"="Proline", "S"="Serine",

```

```

    "T"="Threonine", "W"="Tryptophan", "Y"="Tyrosine", "V"="Valine",
    "*"="Stop codon"
  )

  # Split sequence into individual letters
  letters <- strsplit(toupper(aa_sequence), "")[[1]]

  # Translate each letter to its full name
  full_names <- sapply(letters, function(a) aa_names[[a]])

  return(full_names)
}

```

Let's try them all together (for fun again)

```
seq1 <- generate_dna(60)
```

Well done you!

```
seq1
```

```
[1] "ATGGAGGTCATCAGGAGTTCTCGGAAACAAGAGACTCCAGGGCAGGATCTGAAAGAACCT"
```

```
ami1 <- translate_seq(seq1)
ami1
```

```
[1] "MEVIRSSRKQETPGQDLKEP"
```

```
translate_aminoacids(ami1)
```

M	E	V	I	R
"Methionine"	"Glutamic acid"	"Valine"	"Isoleucine"	"Arginine"
S	S	R	K	Q
"Serine"	"Serine"	"Arginine"	"Lysine"	"Glutamine"
E	T	P	G	Q
"Glutamic acid"	"Threonine"	"Proline"	"Glycine"	"Glutamine"
D	L	K	E	P
"Aspartic acid"	"Leucine"	"Lysine"	"Glutamic acid"	"Proline"

A more advanced example

Make a third function that generates protein sequence of a user specified length and format.

```
generate_prot <- function(x, fasta = T, fullname = F){
  aa_names <- list(
    "A"="Alanine", "R"="Arginine", "N"="Asparagine", "D"="Aspartic acid",
    "C"="Cysteine", "Q"="Glutamine", "E"="Glutamic acid", "G"="Glycine",
    "H"="Histidine", "I"="Isoleucine", "L"="Leucine", "K"="Lysine",
    "M"="Methionine", "F"="Phenylalanine", "P"="Proline", "S"="Serine",
    "T"="Threonine", "W"="Tryptophan", "Y"="Tyrosine", "V"="Valine")
  #Only outputs the symbol of amino acid
  simpleoutput <- sample(names(aa_names), x, T)
  #Renders the full name of amino acid
  fulloutput <- sample(aa_names, x, T)

  if(fasta == T & fullname == F){
    paste(simpleoutput, collapse = "")
  } else if(fasta == T & fullname == T){
    paste(fulloutput, collapse = " --> ")
  } else {
    fulloutput
  }
}
```

Try this out

```
generate_prot(100)
```

```
[1] "TWFKPMANRMLLLHMQHEDTGGRYGPCCAQVVISVRHKFHQYYAVIRQWGIESKQTTGINNGWVGAYMYCLCKLTRQINNIRDSYH
```

Q. Generate a random protein sequences between lengths 5 and 12 amino-acids.

One approach is to do this by brute force calling our function for each length 5 to 12.

Another approach is to write a `for()` loop to iterate over the input valued 5 to 12.

A very useful third R specific approach is to use the `sapply()` function.

```
seq_lengths <- 5:12
for (i in seq_lengths){
  cat(">", i, "\n")
  cat(generate_prot(i))
}
```

```
cat("\n")  
}
```

```
> 5  
QLQGD  
> 6  
VECNQL  
> 7  
LDYMTDK  
> 8  
HADKHKQG  
> 9  
LKSSPAAAT  
> 10  
QACNMLQWTW  
> 11  
IAQTAHRNNVY  
> 12  
DVIHHMWTGSYR
```

```
sapply(5:12, generate_prot)
```

```
[1] "NSPSQ"      "QFVAAP"      "QRWNPLP"      "NFVSNHGH"      "YCNPMYDAL"  
[6] "HNYRFFVFAV" "CGDWAVKEPDH" "FRTNTGNFDMWQ"
```

Key-Point : Writing functions in R is doable but not the easiest thing. Starting with a working snippet of code and then using LLM tools to improve and generalize your function.